Experiment T19: Crack a SHA-1 Hash using John the Ripper and Hashcat

Jyothis Pramod

October 2026

1 Aim

To crack SHA-1 hashes using John the Ripper and Hashcat in an isolated lab environment, and to document the cracked result alongside a speed comparison with tasks T17 and T18.

2 Course Outcome and Module

Course Outcome: Analyze cryptographic vulnerabilities and apply password-cracking techniques using industry-standard penetration testing tools.

Module: Cryptography and System Security.

3 Tools and Versions

Tool	Version	Operating System
John the Ripper	1.9.0-jumbo	Kali Linux
hashcat	6.2.6	Kali Linux

4 Program or Commands

John the Ripper:

```
john --format=raw-sha1 --increment hash.txt
john --show --format=raw-sha1 hash.txt
```

Hashcat:

```
hashcat -m 100 -a 3 hash.txt -1 ?l?d ?1?1?1?1
```

5 How to Run

1. Create a text file named `hash.txt` containing the target SHA-1 hash.

2. Open a terminal in Kali Linux.

3. To crack using John the Ripper, execute: `john --format=raw-sha1 --increment hash.txt`

4. To view the cracked password from John the Ripper, execute: `john --show --format=raw-sha1 hash.txt`

5. To run a mask attack using hashcat, execute: `hashcat -m 100 -a 3 hash.txt -1 ?l?d ?1?1?1?1`. This uses a custom charset of lowercase letters and digits.

6. Record the execution time shown in the terminal output to perform the speed comparison.

6 Sample Output with Screenshots

```
┌──(kali㉿kali)-[~/Desktop]
└─$ john --format=raw-sha1 --increment hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 128/128 SSE2 4x])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password         (?)
1g 0:00:00:00 DONE (2026-10-22 06:52) 1.086g/s 444943p/s 444943c/s 444943C/s password..passwood
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.
```

Figure T19-1: John the Ripper successful cracking output.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ hashcat -m 100 -a 3 hash.txt -1 ?l?d ?1?1?1?1
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
==========================================================================================================================================
* Device #1: cpu-penryn-11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz, 1791/3647 MB (512 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Brute-Force
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.
```

Figure T19-2: Hashcat mask attack initialization.

```
ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Approaching final keyspace - workload adjusted.

Session..........: hashcat
Status...........: Exhausted
Hash.Mode........: 100 (SHA1)
Hash.Target......: 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
Time.Started.....: Thu Oct 22 06:59:33 2026 (1 sec)
Time.Estimated...: Thu Oct 22 06:59:34 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.......: ?1?1?1?1 [4]
Guess.Charset....: -1 ?l?d, -2 Undefined, -3 Undefined, -4 Undefined
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:  4439.7 kH/s (1.02ms) @ Accel:256 Loops:36 Thr:1 Vec:4
Recovered........: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.........: 1679616/1679616 (100.00%)
Rejected.........: 0/1679616 (0.00%)
Restore.Point....: 46656/46656 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-36 Iteration:0-36
Candidate.Engine.: Device Generator
Candidates.#1....: s2xv → xqxv
Hardware.Mon.#1..: Util: 72%

Started: Thu Oct 22 06:59:29 2026
Stopped: Thu Oct 22 06:59:36 2026

┌──(kali㉿kali)-[~/Desktop]
└─$ john --format=raw-sha1 --increment hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 128/128 SSE2 4x])
No password hashes left to crack (see FAQ)
```

Figure T19-3: Hashcat mask attack completion metrics.

7 Result

The raw SHA-1 hash was successfully processed using both tools on a local machine. John the Ripper cracked the test hash (password: password) instantly, with a processing speed of 444,943 c/s. Hashcat successfully exhausted the specified 4-character alphanumeric search space in 1 second, reaching a speed of 4,439.7 kH/s.

8 How This Version Differs from the Other Versions of the Same Experiment

Feature	T19 (Current Version)	T18 (Previous Version)
Language/Tool	John the Ripper and Hashcat	Bash shell scripting
Execution overhead	Highly optimized internal processing (C/OpenCL)	External process creation for each candidate
Performance	Extremely fast (e.g., 444,943 c/s)	Slower runtime (e.g., 2 seconds for 310 attempts)

9 Viva

1. What does the --format=raw-sha1 flag do in John the Ripper?

It explicitly tells the tool that the input hashes are raw SHA-1 hashes, preventing it from wasting time auto-detecting the algorithm.

2. What is the purpose of the --increment flag in John the Ripper?

It enables incremental mode, a brute-force mode that tries all possible character combinations.

3. In the hashcat command, what does -m 100 signify?

The -m flag specifies the hash type. In hashcat, 100 corresponds to the raw SHA-1 algorithm.

4. What does the -a 3 flag signify in the hashcat command?

It specifies attack mode 3, a brute-force/mask attack that uses the appended mask to guess the plaintext.

5. Why use a custom charset like -1 ?l?d?

It restricts the search space to a defined set of characters (lowercase letters and digits in this case), which optimizes the attack and prevents wasted guesses on unlikely symbols.

10 Common Errors

Message	Cause	Fix
No password hashes left to crack (see FAQ)	John the Ripper has already cracked this hash in a previous run and saved it to its log.	Use the --show command to view the password, or delete the john.pot file to force a re-run.
Custom-charset 1 is undefined	The command uses a mask like ?1 but fails to define what the custom charset represents.	Add the -1 flag followed by the character set (e.g., -1 ?l?d) before the mask.

Message	Cause	Fix
Not enough allocatable device memory	Hashcat is attempting to allocate more memory than the OpenCL device allows for the attack.	Append -O to use optimized backend kernels, or -w 1 for a low workload profile.

AI assistance declaration: AI assistance was used to help prepare and format this report according to the prescribed chapter structure. The commands were executed and the outputs recorded in Kali Linux.